SAP HYBRIS

Modulegen: A command used to generate a new extension (module/project structure) quickly using templates in SAP Hybris (SAP Commerce).

It generates a training module with extensions like backoffice, core, storefront, fulfillment, facades, and initialdata**,** and these extensions are then added to the localextensions.xml file.

Items.xml: It is used to define what data you want to store in Hybris.

itemtype → Defines a custom model or database table
relation → Defines relationships between two item types
enumtype → Defines a set of fixed constant values
collectiontype → Defines a list or set of values
maptype → Defines key-value pair data structure

Example: Brand2ProductRel and Brand2VehicleRel define one-to-many relations where one Brand links to many Products and Vehicles.

Brand is a custom item with code, name, and country, while Vehicle is an abstract parent with attributes like hp, code, name, cc, and topSpeed.

Car and Bike extend Vehicle, inheriting its attributes.

VehicleJob is used for background jobs, VehicleSavingProcess and OrderTrackingEmailProcess are used for business processes, with fullname as a dynamic attribute calculated at runtime.

```
<relations>
    <relation code="Brand2ProductRel" localized="false" generate="true" autocreate="true">
      <sourceElement type="Brand" qualifier="brand" cardinality="one">
        <modifiers read="true" write="true" search="true"/>
      </sourceElement>
      <targetElement type="Product" qualifier="products" cardinality="many">
        <modifiers read="true" write="true" search="true" optional="true"/>
      </targetElement>
    </relation>
    <relation code="Brand2VehicleRel" localized="false" generate="true" autocreate="true">
      <sourceElement type="Brand" qualifier="brand" cardinality="one">
        <modifiers read="true" write="true" search="true"/>
      </sourceElement>
      <targetElement type="Vehicle" qualifier="vehicles" cardinality="many">
        <modifiers read="true" write="true" search="true" optional="true"/>
      </targetElement>
    </relation>
  </relations>

  <itemtypes>
    <itemtype code="VehicleJob" extends="ServicelayerJob" autocreate="true" generate="true"/>
    <itemtype code="Brand" generate="true" autocreate="true">
      <deployment table="Brands" typecode="15001"/>
      <attributes>
        <attribute qualifier="code" type="java.lang.String">
          <modifiers optional="false" unique="true" initial="true"/>
          <persistence type="property"/>
        </attribute>
        <attribute qualifier="name" type="java.lang.String">
          <persistence type="property"/>
        </attribute>
        <attribute qualifier="country" type="java.lang.String">
```

```
            <persistence type="property"/>
        </attribute>
    </attributes>
</itemtype>

<itemtype code="Vehicle" abstract="true" generate="true" autocreate="true">
    <deployment table="Vehicles" typecode="15002"/>
    <attributes>
        <attribute qualifier="hp" type="java.lang.Integer">
            <modifiers optional="true" unique="false"/>
            <persistence type="property"/>
        </attribute>
        <attribute qualifier="code" type="java.lang.String">
            <modifiers optional="false" unique="true" initial="true"/>
            <persistence type="property"/>
        </attribute>
        <attribute qualifier="name" type="java.lang.String">
            <modifiers optional="false" unique="true" initial="true"/>
            <persistence type="property"/>
        </attribute>
        <attribute qualifier="cc" type="java.lang.Integer">
            <modifiers optional="true" unique="false" initial="true"/>
            <persistence type="property"/>
        </attribute>
        <attribute qualifier="topSpeed" type="java.lang.Integer">
            <modifiers optional="false" unique="false" initial="true"/>
            <persistence type="property"/>
        </attribute>
    </attributes>
</itemtype>
<itemtype code="Car" extends="Vehicle" generate="true" autocreate="true">
</itemtype>
<itemtype code="Bike" extends="Vehicle" generate="true" autocreate="true">
</itemtype>
<itemtype code="VehicleSavingProcess" extends="BusinessProcess">
    <attributes>
        <attribute qualifier="vehicle" type="Item">
            <modifiers optional="true"/>
            <persistence type="property"/>
        </attribute>
    </attributes>
</itemtype>

<itemtype code="OrderTrackingEmailProcess"
          extends="StoreFrontCustomerProcess"
          generate="true"
          autocreate="true">
    <attributes>

        <attribute qualifier="email" type="java.lang.String">
            <persistence type="property"/>
        </attribute>
        <attribute qualifier="name" type="java.lang.String">
            <persistence type="property"/>
        </attribute>
        <attribute qualifier="fullname" type="java.lang.String">
            <persistence type="dynamic" attributeHandler="FullNameHandler"/>
        </attribute>
        <attribute qualifier="frontendTemplateName" type="java.lang.String">
            <persistence type="property"/>
        </attribute>
    </attributes>
</itemtype>
```

Flexible Search: FlexibleSearch in SAP Hybris (SAP Commerce) is a query language used to fetch data from the database using item types instead of table names.

EX :

1. SELECT {pk} FROM {Brand}
2. SELECT {pk} FROM {Brand} WHERE {code} = 'BMW'
3. SELECT {p.pk} FROM {Product AS p JOIN Brand2ProductRel AS rel ON {p.pk} = {rel.target}} WHERE {rel.source} = ?brandPk

CronJobs: Used to run background tasks/jobs automatically or manually**.**

Example:

```
package org.training.core.job;

import de.hybris.platform.cronjob.enums.CronJobResult;
import de.hybris.platform.cronjob.enums.CronJobStatus;
import de.hybris.platform.cronjob.model.CronJobModel;
import de.hybris.platform.processengine.BusinessProcessService;
import de.hybris.platform.servicelayer.cronjob.AbstractJobPerformable;
import de.hybris.platform.servicelayer.cronjob.PerformResult;
import de.hybris.platform.servicelayer.i18n.CommonI18NService;
import de.hybris.platform.servicelayer.model.ModelService;
import de.hybris.platform.site.BaseSiteService;
import de.hybris.platform.store.services.BaseStoreService;
import org.training.core.model.OrderTrackingEmailProcessModel;

import javax.annotation.Resource;
import java.util.logging.Logger;

public class OrderTrackingJobPerformable extends AbstractJobPerformable<CronJobModel>
{
   private static final Logger LOG =
         Logger.getLogger(OrderTrackingJobPerformable.class.getName());
   @Resource
   private BaseSiteService baseSiteService;
   @Resource
   private BaseStoreService baseStoreService;
   @Resource
   private CommonI18NService commonI18NService;
   @Resource
   private ModelService modelService;
   @Resource
   private BusinessProcessService businessProcessService;
   @Override
   public PerformResult perform(CronJobModel cronJobModel)
   {
      LOG.info("Performing order tracking job");
      OrderTrackingEmailProcessModel process =
            businessProcessService.createProcess(
                  "trackingEmailProcess-" + System.currentTimeMillis(),
                  "orderTrackingProcess");
      process.setEmail("ganeshgana939@gmail.com");
      process.setName("Order Tracking");
      process.setSite(baseSiteService.getBaseSiteForUID("apparel-uk"));
      process.setStore(baseStoreService.getBaseStoreForUid("apparel-uk"));
      process.setLanguage(commonI18NService.getLanguage("en"));
      process.setCurrency(commonI18NService.getCurrency("GBP"));
      LOG.info("*******************************************************");
      LOG.info(process.getFullname());
      LOG.info("*******************************************************");

      process.setFrontendTemplateName("orderTrackingEmailTemplate");
```

```
        modelService.save(process);
        businessProcessService.startProcess(process);

        return new PerformResult(CronJobResult.SUCCESS, CronJobStatus.FINISHED);
    }
    public BusinessProcessService getBusinessProcessService()
    {
        return businessProcessService;
    }
    public void setBusinessProcessService(final BusinessProcessService businessProcessService)
    {
        this.businessProcessService = businessProcessService;
    }
}
```

Bean :
```
<bean id="orderTrackingJobPerformable" class="org.training.core.job.OrderTrackingJobPerformable"
    parent="abstractJobPerformable">
    <property name="modelService" ref="modelService"/>
    <property name="businessProcessService" ref="businessProcessService"/>
</bean>
```

The above class is a CronJob in SAP Hybris (SAP Commerce) used to trigger an email business process. The perform() method creates a new OrderTrackingEmailProcess and sets details like email, site, store, language, and currency.

It saves the process using ModelService and starts it using BusinessProcessService. Finally, it logs details and returns success after triggering the email workflow.

ImpEx: impEx in SAP Hybris (SAP Commerce) is used to import and export data into the system. It allows you to insert, update, or remove data using a structured script format. It works with item types and attributes defined in items.xml. It is mainly used for data setup, migration, and bulk data operations.

1. **Create Job** : INSERT_UPDATE ServicelayerJob;code[unique=true];springId

   ;orderTrackingJob;orderTrackingJobPerformable

2. **Create CronJob :** INSERT_UPDATE CronJob;code[unique=true];job(code);singleExecutable

   ;orderTrackingCronJob;orderTrackingJob;false

3. **Create Trigger (Optional):** INSERT_UPDATE Trigger;cronJob(code)[unique=true];cronExpression

   ;orderTrackingCronJob;0 0/5 * * * ?

ImpEx Decorator → Used to modify or customize data during import

```
package org.training.core.decorators;

import de.hybris.platform.impex.jalo.header.AbstractImpExCSVCellDecorator;

import java.util.Map;
import java.util.logging.Logger;

public class SuffixDecorator extends AbstractImpExCSVCellDecorator {
    private static final Logger LOG = Logger.getLogger(SuffixDecorator.class.getName());
    @Override
    public String decorate(int i, Map<Integer, String> map) {
        String value = map.get(i);
```

```
        if(value==null)
            value = "";
        LOG.info("****************************** Decorator Called ******************************");
        return value+"outs";
    }
}
```

Bean : <bean id="suffixDecorator" class="org.training.core.decorators.SuffixDecorator" />

This class is an ImpEx Decorator in SAP Hybris (SAP Commerce). It modifies the imported data before saving into the system. The decorate() method takes the value and appends "outs" to it. If the value is null, it converts it to empty and then adds the suffix.

ImpEx Translator → Used to convert data from one format to another during import/export

```
package org.training.core.translator;

import de.hybris.platform.impex.jalo.translators.AbstractValueTranslator;
import de.hybris.platform.jalo.Item;
import de.hybris.platform.jalo.JaloInvalidParameterException;
import org.training.core.jalo.OrderTrackingEmailProcess;

import java.util.logging.Logger;

public class NameToUpperCase extends AbstractValueTranslator {
    private static final Logger LOG = Logger.getLogger(NameToUpperCase.class.getName());
    @Override
    public Object importValue(String s, Item item) throws JaloInvalidParameterException {
        LOG.info("****************************** Translator import Called ******************************");
        return s.toUpperCase();
    }

    @Override
    public String exportValue(Object o) throws JaloInvalidParameterException {
        LOG.info("****************************** Translator Export Called ******************************");
        return o.toString().toLowerCase();
    }
}
```

Bean : <bean id="nameToUpperCase" class="org.training.core.decorators.NameToUpperCase" />

This class is an ImpEx Translator in SAP Hybris (SAP Commerce). It converts data during import and export operations. During import, it changes the value to uppercase. During export, it converts the value to lowercase.

ImpEx Export → Used to extract data from Hybris system into files (like CSV)

INTERCEPTORS: Interceptors in SAP Hybris (SAP Commerce) are used to add custom logic during the lifecycle of a model, such as when it is created, updated, loaded, or deleted. They are automatically triggered by the system and help in validating data, setting default values, or modifying data before it is saved. Common types include ValidateInterceptor, PrepareInterceptor, InitDefaultsInterceptor, LoadInterceptor, and RemoveInterceptor, each serving a specific purpose in handling data processing.

1. **InitDefaultsInterceptor** :

```
package org.training.core.interceptor;

import de.hybris.platform.servicelayer.interceptor.InitDefaultsInterceptor;
import de.hybris.platform.servicelayer.interceptor.InterceptorContext;
import org.training.core.model.BikeModel;
import org.training.core.model.CarModel;
```

```
import org.training.core.model.VehicleModel;

public class VehicleInitDefaultsInterceptor implements InitDefaultsInterceptor<VehicleModel> {
   public void onInitDefaults(VehicleModel vehicle, InterceptorContext context){
      if(vehicle instanceof BikeModel bike){
         bike.setCc(200);
         bike.setHp(24);
      }
      if(vehicle instanceof CarModel car){
         car.setCc(200);
         car.setHp(24);
      }
   }
}
```

Bean :
```
<bean id="vehicleInitDefaultsInterceptors" class="org.training.core.interceptor. VehicleInitDefaultsInterceptors "/>
    <bean class="de.hybris.platform.servicelayer.interceptor.impl.InterceptorMapping">
   <property name="interceptor" ref=" vehicleInitDefaultsInterceptors "/>
  <property name="typeCode" value="Vehicle"/>
 </bean>
```

This class is an InitDefaultsInterceptor in SAP Hybris (SAP Commerce) used to set default values when a Vehicle is created. It checks whether the object is a Bike or Car and assigns default values for cc and hp. This interceptor runs automatically during object creation and ensures initial values are set before saving.

2. **Prepare Interceptor**:

```
package org.training.core.interceptor;

import de.hybris.platform.servicelayer.interceptor.InterceptorContext;
import de.hybris.platform.servicelayer.interceptor.PrepareInterceptor;
import org.training.core.model.BikeModel;
import org.training.core.model.CarModel;
import org.training.core.model.VehicleModel;

import java.util.logging.Logger;

public class VehiclePrepareInterceptor implements PrepareInterceptor<VehicleModel> {
   public static final Logger LOG = Logger.getLogger(VehiclePrepareInterceptor.class.getName());
   public void onPrepare(VehicleModel vehicle, InterceptorContext context){
      if(vehicle instanceof BikeModel bike){
         bike.setCode(bike.getCode().toUpperCase());
         LOG.info("Preparing Bike:" + bike.getName());
      }
      if(vehicle instanceof CarModel car){
         car.setCode(car.getCode().toUpperCase());
         LOG.info("Preparing Car:" + car.getName());
      }
   }
}
```

Bean :
```
<bean id="vehiclePrepareInterceptor" class="org.training.core.interceptor.VehiclePrepareInterceptor"/>
    <bean class="de.hybris.platform.servicelayer.interceptor.impl.InterceptorMapping">
   <property name="interceptor" ref="vehiclePrepareInterceptor"/>
  <property name="typeCode" value="Vehicle"/>
 </bean>
```

This class is a PrepareInterceptor in SAP Hybris (SAP Commerce) used to modify data before saving. It checks whether the object is a Bike or Car and converts the code to uppercase. It also logs the vehicle name during preparation. This interceptor runs automatically before the model is saved to the database.

3. Validate Interceptor:

```
package org.training.core.interceptor;
import de.hybris.platform.servicelayer.interceptor.InterceptorContext;
import de.hybris.platform.servicelayer.interceptor.InterceptorException;
import de.hybris.platform.servicelayer.interceptor.ValidateInterceptor;
import org.training.core.model.BikeModel;
import org.training.core.model.CarModel;
import org.training.core.model.VehicleModel;
public class VehicleValidateInterceptor implements ValidateInterceptor<VehicleModel> {
   public void onValidate(VehicleModel vehicle,InterceptorContext context) throws InterceptorException{
      if(vehicle instanceof BikeModel bike){
         if(bike.getBrand()==null){
            throw new InterceptorException("Brand must be Exist!!");
         }
         if(bike.getName()==null){
            throw new InterceptorException("Name must be Exist!!");
         }
         if(bike.getCode()==null){
            throw new InterceptorException("Code must be Exist!!");
         }
      }
      if(vehicle instanceof CarModel car){
         if(car.getBrand()==null){
            throw new InterceptorException("Brand must be Exist!!");
         }
         if(car.getName()==null){
            throw new InterceptorException("Name must be Exist!!");
         }
         if(car.getCode()==null){
            throw new InterceptorException("Code must be Exist!!");
         }
      }
   }
}
```

Bean :
```
<bean id="vehicleValidateInterceptor " class="org.training.core.interceptor.VehicleValidateInterceptor "/>
   <bean class="de.hybris.platform.servicelayer.interceptor.impl.InterceptorMapping">
   <property name="interceptor" ref="vehicleValidateInterceptor "/>
   <property name="typeCode" value="Vehicle"/>
 </bean>
```

This class is a ValidateInterceptor in SAP Hybris (SAP Commerce) used to validate data before saving. It checks whether Bike or Car has required fields like brand, name, and code. If any of these fields are missing, it throws an exception. This interceptor ensures only valid data is saved in the system.

4. Loading Interceptor:

```
package org.training.core.interceptor;

import de.hybris.platform.servicelayer.interceptor.InterceptorContext;
import de.hybris.platform.servicelayer.interceptor.InterceptorException;
import de.hybris.platform.servicelayer.interceptor.LoadInterceptor;

import org.training.core.model.VehicleModel;

import java.util.logging.Logger;

public class VehicleLoadingInterceptor implements LoadInterceptor<VehicleModel>
{
   public static final Logger LOG = Logger.getLogger(VehicleLoadingInterceptor.class.getName());
   @Override
   public void onLoad(final VehicleModel vehicleModel, final InterceptorContext ctx) throws InterceptorException
```

```
    {
      if(vehicleModel.getName() != null)
      {
        LOG.info("Vehicle Loaded: " + vehicleModel.getName());
      }
    }
}
```

Bean :
```
<bean id="vehicleLoadingInterceptor " class="org.training.core.interceptor.VehicleLoadingInterceptor "/>
    <bean class="de.hybris.platform.servicelayer.interceptor.impl.InterceptorMapping">
      <property name="interceptor" ref="vehicleLoadingInterceptor "/>
      <property name="typeCode" value="Vehicle"/>
    </bean>
```

This class is a LoadInterceptor in SAP Hybris (SAP Commerce) used to execute logic when a Vehicle is loaded from the database. It checks if the vehicle name is not null and logs the name. This interceptor runs automatically whenever the model is fetched.

5. **Remove Interceptor** :

```
package org.training.core.interceptor;

import de.hybris.platform.servicelayer.interceptor.RemoveInterceptor;
import de.hybris.platform.servicelayer.interceptor.InterceptorContext;
import de.hybris.platform.servicelayer.interceptor.InterceptorException;

import org.training.core.model.BrandModel;

public class BrandRemoveInterceptor implements RemoveInterceptor<BrandModel>
{

    @Override
    public void onRemove(final BrandModel brand, final InterceptorContext context)
        throws InterceptorException
    {
      if (brand.getVehicles() != null && !brand.getVehicles().isEmpty())
      {
        throw new InterceptorException("Cannot delete brand because vehicles exist.");
      }
    }
}
```

Bean :
```
<bean id="brandRemoveInterceptor " class="org.training.core.interceptor.BrandRemoveInterceptor "/>
    <bean class="de.hybris.platform.servicelayer.interceptor.impl.InterceptorMapping">
      <property name="interceptor" ref="brandRemoveInterceptor "/>
      <property name="typeCode" value="Vehicle"/>
    </bean>
```

This class is a RemoveInterceptor in SAP Hybris (SAP Commerce) used to control deletion of data. It checks if the Brand has associated Vehicles before removing. If vehicles exist, it throws an exception. This prevents deletion of a Brand that is still in use.

EVENTS AND LISTENERS : Events and Listeners in SAP Hybris (SAP Commerce) are used for asynchronous communication between components. An event is triggered when something happens in the system, and a listener listens to that event and performs some action. This helps in decoupling logic, so different parts of the application can react independently. They are commonly used for tasks like sending emails, updating data, or triggering business processes.

EVENT :

```
package org.training.core.event.myevents;
import de.hybris.platform.core.PK;
import de.hybris.platform.servicelayer.event.events.AbstractEvent;

import java.util.logging.Level;
import java.util.logging.Logger;

public class BikeSavingEvent extends AbstractEvent {
   private final PK pk;
   private static final Logger LOG = Logger.getLogger(BikeSavingEvent.class.getName());
   public BikeSavingEvent(PK pk){
      this.pk=pk;
   }
   public void display(){
      LOG.log(Level.INFO, "Bike Saving Event: {0}", pk);
   }
}
```

This class is a custom Event in SAP Hybris used to represent a Bike saving action. It extends AbstractEvent and carries the PK of the Bike as data. The display() method logs the event information. This event can be triggered and handled by listeners for further processing.

LISTENER :

```
package org.training.core.listeners;

import de.hybris.platform.servicelayer.event.impl.AbstractEventListener;
import org.training.core.event.myevents.BikeSavingEvent;

public class BikeSaveListener extends AbstractEventListener<BikeSavingEvent> {
   @Override
   public void onEvent(BikeSavingEvent event) {
      event.display();
   }
}
```

This class is an Event Listener in SAP Hybris (SAP Commerce) used to handle BikeSavingEvent. It listens for the event and executes logic when the event is triggered. The onEvent() method calls the display() method of the event. This helps process the event asynchronously.

BikeSavingEvent is triggered manually using EventService from service, facade, controller, or interceptor.

```
eventService.publishEvent(new BikeSavingEvent(bike.getPk()));
```

STOREFRONT API: Storefront API in SAP Hybris (SAP Commerce) refers to the controller-based layer that connects the frontend UI with backend logic. It handles user requests from the browser, calls facades/services to get data, and sends the response to the view (JSP pages). These APIs are built using Spring MVC controllers and are mainly used for rendering web pages like products, carts, and orders.

Example:

```
package org.training.storefront.controllers.pages;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.training.facades.VehicleFacade;

import javax.annotation.Resource;
import java.util.logging.Logger;

@Controller
@RequestMapping(value = "/vehicle")
public class VehicleController
{
   private  static final Logger LOG = Logger.getLogger(VehicleController.class.getName());
   @Resource
   private VehicleFacade vehicleFacade;
   @GetMapping
   public String getVehicles(Model model)
   {
      LOG.info("*****Inside Controller*******");
      model.addAttribute("vehicles", vehicleFacade.getVehicles());
      LOG.info("*******"+vehicleFacade.getVehicles().toString());
      return "pages/vehicle/vehiclePage";
   }
}
```

This class is a Storefront Controller in SAP Hybris (SAP Commerce) used to handle web requests. It listens to the URL /vehicle and calls the VehicleFacade to get vehicle data. The data is added to the model and sent to the view page. Finally, it returns the JSP page to display the vehicles on the storefront.

BUSINESS PROCESS: Business Process in SAP Hybris (SAP Commerce) is used to automate workflows like email sending, order processing, and notifications. It works based on defined steps (actions) and transitions configured in process XML files. The process is triggered from code (like a CronJob or service) and runs in the background. It helps manage complex operations in a structured and asynchronous way.

myOrderTrackingEmailProcess.xml (trainingcore\resources\trainingcore\processes)

```
<process xmlns="http://www.hybris.de/xsd/processdefinition"
      start="generateEmail"
      name="orderTrackingProcess"
      processClass="org.training.core.model.OrderTrackingEmailProcessModel">

   <contextParameter name="frontendTemplateName"
               use="optional"
               type="java.lang.String"/>
   <action id="generateEmail" bean="myGenerateEmail">
      <transition name="OK" to="sendEmail"/>
      <transition name="NOK" to="failed"/>
   </action>

   <action id="sendEmail" bean="sendEmail">
      <transition name="OK" to="success"/>
      <transition name="NOK" to="failed"/>
   </action>

   <end id="success" state="SUCCEEDED"/>
   <end id="failed" state="FAILED"/>

</process>
```

MyGenerateEmail.class:

```
package org.training.core.actions;

import de.hybris.platform.acceleratorservices.process.email.actions.GenerateEmailAction;
import de.hybris.platform.processengine.model.BusinessProcessModel;
import de.hybris.platform.servicelayer.model.ModelService;
import org.training.core.model.OrderTrackingEmailProcessModel;

import javax.annotation.Resource;
import java.util.logging.Logger;

public class MyGenerateEmail extends GenerateEmailAction {

   public static final Logger LOG = Logger.getLogger(MyGenerateEmail.class.getName());

   @Resource
   private ModelService modelService;

   @Override
   public Transition executeAction(final BusinessProcessModel process) {
      OrderTrackingEmailProcessModel orderTrackingEmailProcessModel =
            (OrderTrackingEmailProcessModel) process;

      LOG.info("*********************************************************");
      LOG.info("Generating Email !!!!!!!!!!!!");
      LOG.info(orderTrackingEmailProcessModel.getFullname());
      LOG.info(orderTrackingEmailProcessModel.getFrontendTemplateName());
      LOG.info("*********************************************************");

      orderTrackingEmailProcessModel.setEmail("ganeshgana939@gmail.com");
      orderTrackingEmailProcessModel.setFrontendTemplateName("orderTrackingEmailTemplate");
      modelService.save(orderTrackingEmailProcessModel);

      return super.executeAction(orderTrackingEmailProcessModel);
   }
}
```

Bean :

```
<bean id="myGenerateEmail" class="org.training.core.actions.MyGenerateEmail" parent="abstractGenerateEmailAction"/>
  <bean id="sentEmail" class="org.training.core.actions.SentEmail"/>
   <bean id="orderTrackingProcessDefinitionResource"
       class="de.hybris.platform.processengine.definition.ProcessDefinitionResource">
      <property name="resource"
            value="classpath:/trainingcore/processes/myOrderTrackingEmailProcess.xml"/>
   </bean>
```

This is a Business Process in SAP Hybris (SAP Commerce) used to send an order tracking email.

The process starts with generateEmail, which calls your custom action MyGenerateEmail to prepare email data. If successful, it moves to sendEmail to send the email; otherwise, it goes to failed.

In MyGenerateEmail, it logs details, sets email and template values, saves the process, and then calls the default email generation logic.

Finally, the process ends with success (SUCCEEDED) or failed (FAILED) based on execution.

CronJob → createProcess("orderTrackingProcess") → process.xml → actions (generateEmail → sendEmail)

POPULATORS AND CONVERTERS: Populators and Converters in SAP Hybris (SAP Commerce) are used to transform data between different layers of the application. A Converter is responsible for creating the target object from the source model, while a Populator fills the target object with data from the source. They are mainly used in the facade layer to convert model objects into data (DTO) objects, helping to keep the code clean, reusable, and well-structured.

Populator:

```
package org.training.facades.populators;

import de.hybris.platform.converters.Populator;
import de.hybris.platform.servicelayer.dto.converter.ConversionException;
import org.training.core.model.OrderTrackingEmailProcessModel;
import org.training.core.model.VehicleModel;
import org.training.facades.data.VehicleData;

import java.util.logging.Logger;

public class OrderTrackingPopulator implements Populator<OrderTrackingEmailProcessModel, VehicleData> {

   private static final Logger LOG = Logger.getLogger(OrderTrackingPopulator.class.getName());

   @Override
   public void populate(OrderTrackingEmailProcessModel source,
                    VehicleData target) throws ConversionException {

      LOG.info("************* OrderTracking Populator***************");
        target.setEmail(source.getEmail());
        target.setProcessDefinitionName(source.getProcessDefinitionName());
   }
}
```

Bean :

```
<bean id="orderTrackingPopulator"  class="org.training.facades.populators.OrderTrackingPopulator"/>
```

Converter: In Facade,

```
 private Converter<OrderTrackingEmailProcessModel, VehicleData> orderTrackingConverter;
 List<VehicleData> orderDataList = orderTrackingConverter.convertAll(orderTrackings);
```

Bean :

```
<bean id="orderTrackingConverter"
          parent="abstractPopulatingConverter">
          <property name="targetClass"
                    value="org.training.facades.data.VehicleData"/>
          <property name="populators">
                    <list>
                    <ref bean="orderTrackingPopulator"/>
                    </list>
          </property>
</bean>
```

OCC: Omni Commerce Connect (OCC) is the standard RESTful API layer in SAP Commerce Cloud used to expose backend functionality as web services. It allows external applications, such as mobile apps and JavaScript frontends (like Spartacus), to interact with the platform using JSON or XML. By sitting on top of the Facade Layer, it ensures that the same business logic and data transformations are reused across all channels. This "headless" approach decouples the frontend from the backend, enabling a flexible, stateless architecture secured by OAuth2.

Example:

```
package org.training.controllers;
import de.hybris.platform.webservicescommons.mapping.DataMapper;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.Parameter;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.access.annotation.Secured;
import org.springframework.web.bind.annotation.*;
import org.training.dto.VehicleTrackingWsDto;
import org.training.dto.VehicleWsDto;
import org.training.facades.VehicleFacade;
import org.training.facades.data.VehicleData;

import javax.annotation.Resource;
import java.util.List;

@RestController
@RequestMapping("/{baseSiteId}/ordertrack")
@Tag(name = "Vehicle Tracking")
public class OrderTrackingController {

   @Resource(name = "vehicleFacade")
   private VehicleFacade vehicleFacade;

   @Resource(name = "dataMapper")
   private DataMapper dataMapper;
   @Secured({"ROLE_TRUSTED_CLIENT", "ROLE_CUSTOMERGROUP"})
   @GetMapping(value = "/vehicles")
   @ResponseBody
   @Operation(summary = "Get all tracked vehicles")
   public VehicleTrackingWsDto getVehicles(
         @Parameter(description = "Base site identifier", required = true)
         @PathVariable final String baseSiteId,
         @Parameter(description = "Fields: BASIC, DEFAULT, FULL, CUSTOM")
         @RequestParam(required = false, defaultValue = "DEFAULT") final String fields) {
      List<VehicleData> vehicleDataList = vehicleFacade.getVehicles();
      List<VehicleWsDto>  vehicleWsDtoList = dataMapper.mapAsList(vehicleDataList,VehicleWsDto.class,fields);

      VehicleTrackingWsDto wrapper = new VehicleTrackingWsDto();
      wrapper.setVehicles(vehicleWsDtoList);
      return wrapper;
   }
}
```

Bean: **training\customocc\resources\occ\v2\customocc\web\spring\customocc-web-spring.xml**

```
<bean parent="fieldSetLevelMapping" id="TRACKING_VehicleItemMapping">
      <property name="dtoClass" value="org.training.dto.VehicleWsDto"/>
      <property name="levelMapping">
         <map>
            <entry key="BASIC"   value="code,name"/>
            <entry key="DEFAULT" value="name,cc"/>
            <entry key="FULL"    value="code,name,cc,hp,topSpeed"/>
            <entry key="CUSTOM"  value="cc,hp"/>
         </map>
      </property>
   </bean>

   <bean parent="fieldSetLevelMapping" id="TRACKING_VehicleTrackingMapping">
      <property name="dtoClass" value="org.training.dto.VehicleTrackingWsDto"/>
      <property name="levelMapping">
         <map>
```

```
            <entry key="BASIC"   value="vehicles(BASIC)"/>
            <entry key="DEFAULT" value="vehicles(DEFAULT)"/>
            <entry key="FULL"    value="vehicles(FULL)"/>
            <entry key="CUSTOM">
               <value>vehicles(cc,hp)</value>
            </entry>
         </map>
      </property>
   </bean>
```

This OCC (Omni Commerce Connect) controller exposes vehicle tracking data as a RESTful web service, using the @{baseSiteId} variable to ensure site-specific context. It leverages the DataMapper utility to transform backend VehicleData into a VehicleWsDto based on requested field levels like BASIC or FULL. Access is restricted to authorized users via the @Secured annotation, ensuring that only trusted clients or specific customer groups can retrieve the data. Finally, the controller uses Swagger/OpenAPI annotations to automatically generate interactive documentation, making the API easy for frontend developers to integrate.

HOT FOLDER : The Hot Folder is a Spring Integration-based framework in SAP Commerce that enables automated data synchronization by monitoring specific server directories for incoming files. Once a file is detected, it is automatically parsed and transformed into ImpEx format based on predefined mapping configurations. This mechanism is essential for integrating with external systems, as it allows for hands-free, high-volume updates to the database. Processed files are moved to archive or error folders, ensuring a clear audit trail of all automated data imports.

Example : trainingcore\resources\trainingcore\integration\hot-folder-store-training-spring.xml

```
 <?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"

      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

      xmlns:int="http://www.springframework.org/schema/integration"

      xmlns:file="http://www.springframework.org/schema/integration/file"

      xmlns:p="http://www.springframework.org/schema/p"

      xmlns:context="http://www.springframework.org/schema/context"

      xsi:schemaLocation="http://www.springframework.org/schema/integration http://www.springframework.org/schema/integration/spring-
integration.xsd

      http://www.springframework.org/schema/integration/file      http://www.springframework.org/schema/integration/file/spring-integration-
file.xsd

      http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd

      http://www.springframework.org/schema/context

      http://www.springframework.org/schema/context/spring-context.xsd">

   <context:annotation-config/>

   <bean id="baseDirectoryTraining" class="java.lang.String">

       <constructor-arg value="#{baseDirectory}/${tenantId}/training" />
```

```
    </bean>

    <file:inbound-channel-adapter id="batchFilesTraining" directory="#{baseDirectoryTraining}"

                        filename-regex="^(.*)-(\d+)\.csv"

                        comparator="fileOrderComparator">

        <int:poller fixed-rate="1000" />

    </file:inbound-channel-adapter>

    <file:outbound-gateway request-channel="batchFilesTraining" reply-channel="batchFilesTrainingProc"

                    directory="#{baseDirectoryTraining}/processing"

                    delete-source-files="true" />

    <int:service-activator input-channel="batchFilesTrainingProc" output-channel="batchFilesHeaderInit"

                    ref="trainingHeaderSetupTask"

                    method="execute" />

    <bean id="trainingHeaderSetupTask" class="de.hybris.platform.acceleratorservices.dataimport.batch.task.HeaderSetupTask">

        <property name="catalog" value="trainingProductCatalog" />

        <property name="net" value="false" />

        <property name="storeBaseDirectory" ref="baseDirectoryTraining" />

    </bean>

    <bean id="trainingOrderConverterMapping"

         class="de.hybris.platform.acceleratorservices.dataimport.batch.converter.mapping.impl.DefaultConverterMapping"

         p:mapping="OrderTrackingEmailProcess"

         p:converter-ref="trainingOrderConverter"/>

    <beanid="trainingOrderConverter"
class="de.hybris.platform.acceleratorservices.dataimport.batch.converter.impl.DefaultImpexConverter">

        <property name="header">

            <value>

                INSERT_UPDATE OrderTrackingEmailProcess;code[unique=true];name;processDefinitionName

            </value>

        </property>

        <property name="impexRow">
```

```
        <value>;{+0};{1};{2}</value>

      </property>

    </bean>

</beans>
```

This Hot Folder configuration uses Spring Integration to monitor a specific directory for CSV files matching a defined regex every 1000ms. When a file is detected, it is moved to a processing folder and passed to a HeaderSetupTask that assigns it to the trainingProductCatalog. A DefaultImpexConverter then transforms the raw CSV rows into INSERT_UPDATE scripts for the OrderTrackingEmailProcess item type. This setup automates the data lifecycle from file detection to database persistence without manual intervention.

ODATA : OData integration in SAP Commerce Cloud (Hybris) uses Integration Objects to facilitate inbound (data into Hybris) and outbound (data out of Hybris) communication via RESTful APIs. Inbound scenarios use HTTP/OData services to create/update data, while outbound scenarios use Outbound Sync or Outbound Service to send updates (e.g., orders) to external systems (e.g., SAP ERP) in real-time or via schedules.

This bidirectional framework provides a modern, API-first approach to system integration by mapping internal ItemTypes to structured JSON payloads.

By leveraging Integration Objects, it ensures a decoupled and scalable way to synchronize data with external cloud architectures and third-party platforms.

Example :

```
# 1. Integration Object (The Service)
INSERT_UPDATE IntegrationObject; code[unique = true]; integrationType(code)
; OrderEmailService ; INBOUND

# 2. Integration Object Item (The Table)
INSERT_UPDATE IntegrationObjectItem; integrationObject(code)[unique = true]; code[unique = true]; type(code); root[default
= false]
; OrderEmailService ; OrderTrackingEmailProcess ; OrderTrackingEmailProcess ; true

# 3. Attributes (Fixed Header: 'intObjItem' is the standard qualifier)
INSERT_UPDATE IntegrationObjectItemAttribute; integrationObjectItem(integrationObject(code), code)[unique = true];
attributeName[unique = true]; attributeDescriptor(enclosingType(code), qualifier); unique[default = false]
; OrderEmailService:OrderTrackingEmailProcess ; code ; OrderTrackingEmailProcess:code ; true
; OrderEmailService:OrderTrackingEmailProcess ; name ; OrderTrackingEmailProcess:name ; false
; OrderEmailService:OrderTrackingEmailProcess ; email ; OrderTrackingEmailProcess:email ; false
; OrderEmailService:OrderTrackingEmailProcess ; fullname ; OrderTrackingEmailProcess:fullname ; false
; OrderEmailService:OrderTrackingEmailProcess ; processDefinitionName ;
OrderTrackingEmailProcess:processDefinitionName ; false

# 4. Inbound Channel Configuration (Security Mode)
INSERT_UPDATE InboundChannelConfiguration; integrationObject(code)[unique = true]; authenticationType(code)
; OrderEmailService ; OAUTH

# 5. Create Technical User
INSERT_UPDATE Employee; uid[unique = true]; name; groups(uid); backofficeLoginDisabled[default=true]
; tracking_api_user ; Tracking API User ; integrationadmingroup,admingroup ; true

# 6. OAuth Client Details (Fixed Header: Added authorities and resourceIds)
# Without ROLE_CLIENT and ROLE_TRUSTED_CLIENT, the OData filter will return 403.
INSERT_UPDATE IntegrationClientCredentialsDetails; clientId[unique = true]; clientSecret; user(uid); authorities;
authorizedGrantTypes; scope; resourceIds
; tracking_service_client ; secret ; tracking_api_user ;
ROLE_CLIENT,ROLE_TRUSTED_CLIENT,ROLE_INTEGRATIONADMINGROUP ; client_credentials ; extended ; hybris
```

```
---------------------------------------------------------------------------------------------------------------------------------------------------------
# 1. Define the Integration Object (The Data Structure)
INSERT_UPDATE IntegrationObject; code[unique = true]; integrationType(code)
; OrderTrackingOutboundIO ; OUTBOUND

# 2. Define the Item Type (The Model)
INSERT_UPDATE IntegrationObjectItem; integrationObject(code)[unique = true]; code[unique = true]; type(code); root[default
= false]
; OrderTrackingOutboundIO ; OrderTrackingEmailProcess ; OrderTrackingEmailProcess ; true

# 3. Define the Attributes (The Fields)
INSERT_UPDATE IntegrationObjectItemAttribute; integrationObjectItem(integrationObject(code), code)[unique = true];
attributeName[unique = true]; attributeDescriptor(enclosingType(code), qualifier); unique[default = false]
; OrderTrackingOutboundIO:OrderTrackingEmailProcess ; code; OrderTrackingEmailProcess:code  ; true
; OrderTrackingOutboundIO:OrderTrackingEmailProcess ; name  ; OrderTrackingEmailProcess:name; false
; OrderTrackingOutboundIO:OrderTrackingEmailProcess ; email  ; OrderTrackingEmailProcess:email  ; false
; OrderTrackingOutboundIO:OrderTrackingEmailProcess; processDefinitionName
;OrderTrackingEmailProcess:processDefinitionName ; false

# 4. Define the Target Destination (The "Where")
INSERT_UPDATE DestinationTarget; id[unique = true]
; OutboundTestTarget

# 5. Define the Endpoint (Base URL)
INSERT_UPDATE Endpoint; id[unique = true]; version[unique = true]; specUrl; name
; OutboundTestEndpoint ; v1 ; "https://webhook.site" ; External Test API

# 6. Define the Credentials (Optional for testing)
INSERT_UPDATE BasicCredential; id[unique = true]; username; password
; OutboundTestCreds ; admin ; nimda

# 7. Define the Consumed Destination (The Final URL)
# REPLACE the URL with your unique Webhook.site or Postman Mock URL
INSERT_UPDATE ConsumedDestination; id[unique = true]; url; endpoint(id, version); destinationTarget(id); credential(id)
; OutboundTestDestination ; "https://webhook.site/fd2fee1f-8d15-4fa5-9cf4-edb159c4be7d" ; OutboundTestEndpoint:v1 ;
OutboundTestTarget ; OutboundTestCreds

# 8. Create the Outbound Channel Configuration (The Bridge)
INSERT_UPDATE OutboundChannelConfiguration; code[unique = true]; integrationObject(code); destination(id)
; OutboundTestChannel ; OrderTrackingOutboundIO ; OutboundTestDestination
```

This configuration establishes a bidirectional data exchange for order tracking using the SAP Integration Suite (Integration API). It defines Inbound OData services secured by OAuth2 to receive data and Outbound Sync channels to push model updates to external endpoints like webhooks. By mapping specific attributes of the OrderTrackingEmailProcess to Integration Objects, it ensures a decoupled and structured JSON communication. This setup eliminates the need for manual ImpEx imports by providing a modern, API-first approach to system integration.

BACKOFFICE CUSTOMIZATION : Backoffice Customization leverages the CockpitNG framework to tailor the administration console through XML configurations and custom Java components. By modifying the backoffice-config.xml, developers can define specialized **Editor Areas**, **List Views**, and **Explorer Tree** nodes to optimize the business user experience. For advanced requirements, custom **Widgets** and **Actions** can be implemented to trigger complex backend logic directly from the UI. This modular approach ensures that the management interface remains intuitive, scalable, and aligned with unique organizational workflows.

Example : custombackoffice\resources\custombackoffice-backoffice-config.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>

<config xmlns="http://www.hybris.com/cockpit/config"

      xmlns:editorArea="http://www.hybris.com/cockpitng/component/editorArea"

      xmlns:list-view="http://www.hybris.com/cockpitng/component/listView"

      xmlns:advanced-search="http://www.hybris.com/cockpitng/config/advancedsearch"

      xmlns:explorer-tree="http://www.hybris.com/cockpitng/config/explorertree">

   <!-- Advanced Search -->

   <context type="OrderTrackingEmailProcess" component="advanced-search"

          merge-by="type">

      <advanced-search:advanced-search>

         <advanced-search:field-list>

            <advanced-search:field name="code" selected="true"/>

            <advanced-search:field name="email" selected="true"/>

            <advanced-search:field name="fullname" selected="true"/>

            <advanced-search:field name="state" selected="true"/>

         </advanced-search:field-list>

      </advanced-search:advanced-search>

   </context>

   <!-- List View -->

   <context type="OrderTrackingEmailProcess" component="listview"

          merge-by="type">

      <list-view:list-view>

         <list-view:column qualifier="code"/>

         <list-view:column qualifier="email"/>

         <list-view:column qualifier="fullname"/>

         <list-view:column qualifier="state"/>

      </list-view:list-view>
```

```
</context>

<!-- Editor Area -->

<context type="OrderTrackingEmailProcess" component="editor-area"

        merge-by="type">

    <editorArea:editorArea>

        <editorArea:essentials>

            <editorArea:essentialSection name="hmc.essential">

                <editorArea:attribute qualifier="code" readonly="true"/>

                <editorArea:attribute qualifier="email"/>

                <editorArea:attribute qualifier="state" readonly="true"/>

            </editorArea:essentialSection>

        </editorArea:essentials>

        <editorArea:tab name="training.tab.tracking">

            <editorArea:section name="training.section.trackingdetails">

                <editorArea:attribute qualifier="code" readonly="true"/>

                <editorArea:attribute qualifier="email"/>

                <editorArea:attribute qualifier="fullname"/>

                <editorArea:attribute qualifier="frontendTemplateName"/>

                <editorArea:attribute qualifier="state" readonly="true"/>

            </editorArea:section>

        </editorArea:tab>

    </editorArea:editorArea>

</context>

<!-- Explorer Tree -->

<context merge-by="module" component="explorer-tree">

    <explorer-tree:explorer-tree>

        <explorer-tree:navigation-node id="hmc_treenode_order"

                                       merge-mode="append">

            <explorer-tree:type-node id="hmc.ordertracking"
```

```
                    code="OrderTrackingEmailProcess"/>

        </explorer-tree:navigation-node>

      </explorer-tree:explorer-tree>

    </context>

</config>
```

This Backoffice configuration customizes the administration UI for OrderTrackingEmailProcess by defining its searchability, visual layout, and navigation. It establishes a dedicated Explorer Tree node for easy access, a tailored List View for data oversight, and an organized Editor Area with specific read-only restrictions. By utilizing the CockpitNG XML framework, these settings ensure that business users can efficiently manage tracking data through a clean, structured interface. This modular approach allows for seamless UI updates without modifying the underlying Java business logic.